Rilevazione e Mitigazione di un Attacco DoS in una Rete Software-Defined (SDN)

Progetto — Network and Cloud Infrastructures · A.A. 2026/2027

Amodio Martina · Catena Maria · Cennamo Antonio

Mat. DE9000018 · Mat. DE9000020 · Mat. DE9000019

1. Overview del Progetto

Il presente progetto progetta e valida sperimentalmente, tramite l'emulatore Mininet e il controller Ryu, una rete SDN soggetta a un attacco DoS UDP, implementando un meccanismo di rilevazione automatica e mitigazione reattiva con quarantena forense. L'obiettivo è dimostrare la superiorità dell'approccio SDN rispetto a un'architettura priva di difese, sia in termini di throughput dei flussi legittimi che di isolamento e analisi dell'attaccante.

Gli obiettivi specifici verificati sperimentalmente sono:

1. rilevazione automatica del flood UDP in $\approx 6s$ dall'inizio dell'attacco;
2. installazione della regola OpenFlow di mitigazione in meno di $1ms$;
3. ripristino integrale del throughput TCP dei nodi legittimi (h2, h4) da 0 a $1.05\ Mbit/s$ senza interruzione del servizio per il nodo UDP prioritario h5;
4. cattura e analisi forense del traffico malevolo sull'host di quarantena h6 mediante $.pcap$ in tempo reale.

2. Topologia di Rete e Configurazione

La rete è emulata in Mininet su una singola VM Ubuntu 22.04, con una topologia ad albero a due livelli: quattro switch OpenFlow (S1–S4), sei host (h1–h6) e un controller Ryu esterno, collegato agli switch tramite OpenFlow 1.3 su un canale fuori-banda. I link di accesso e quelli S1–S3 e S2–S3 sono a 10 Mbit/s.
Il link S3–S4, collo di bottiglia, ha capacità di 6 Mbit/s, e tra gli switch il ritardo è di 25 ms per rappresentare collegamenti a lunga distanza. Ogni host ha un ruolo preciso:

- *h1* è l'attaccante e genera un flood UDP verso *h3*.
- *h2* e *h4* sono utenti legittimi che trasmettono TCP verso *h3*.
- *h5* è un utente legittimo prioritario che trasmette UDP a banda sostenuta (*2.11 Mbit/s*) e che non deve mai essere bloccato.
- *h3*, collegato a *S4*, è il server.
- *h6*, collegato alla porta 4 di *S1*, è l'host di quarantena che riceve il traffico di *h1* rediretto dal controller.

La topologia è pensata per mettere alla prova aspetti precisi della mitigazione. L'albero privo di anelli consente il forwarding L2 ordinario senza protocolli anti-loop, lasciando al centro dell'esperimento il DoS. Il collo di bottiglia S3–S4 sta dopo il punto in cui i flussi convergono, quindi attacco e traffico legittimo competono per la stessa risorsa. h1 e h4 condividono S1: bloccare l'intera porta S1–S3 colpirebbe anche h4 (il cosiddetto caso *"Piracy Shield"*), perciò la regola deve essere mirata sull'IP dell'attaccante. h5 è un flusso UDP legittimo sopra la soglia: dimostra che il controller non decide solo in base al volume, ma anche tramite whitelist. Infine h6 sta sullo stesso switch di h1, così l'attacco viene intercettato al primo salto, prima del collo di bottiglia, e le prove restano disponibili con le intestazioni originali.

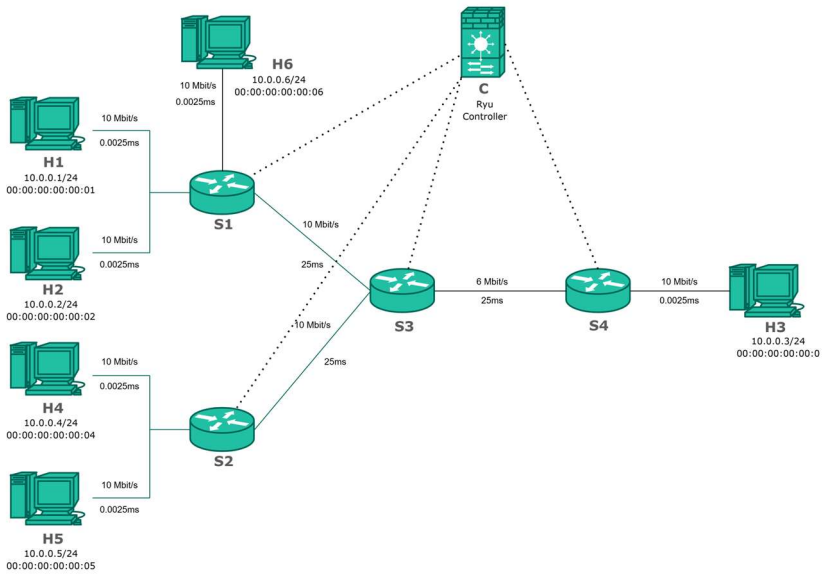


Figura 1 – Topologia di rete: h1 attaccante, h2/h4 utenti TCP, h5 utente UDP prioritario, h3 server, h6 quarantena.

Host	Ruolo	**Traffico generato** tramite iperf
h1	Attaccante	Flood UDP sulla porta h3:5001
h2	Utente TCP legittimo	TCP sulla porta h3:5002
h4	Utente TCP legittimo	TCP sulla porta h3:5003
h5	Utente UDP prioritario (whitelist)	UDP 2.11 Mbit/s sulla porta h3:5004
h3	Server	Server iperf in ascolto sulle porte 5001–5004
h6	Quarantena forense	Riceve il traffico di h1 rediretto; in esecuzione tcpdump che produce un file .pcap

3. Logica del controller SDN e Scelte Progettuali

Il controller è un'applicazione Ryu che estende SimpleSwitch13, da cui eredita il normale apprendimento degli indirizzi MAC, e vi aggiunge un thread di monitoraggio che a intervalli regolari interroga gli switch sul traffico che hanno gestito. Per tenere separate la sicurezza e l'inoltro, ogni switch usa due tabelle OpenFlow: i pacchetti attraversano prima la Tabella 0, dedicata a classificazione e protezione, e poi la Tabella 1, che si occupa dell'inoltro.
Le regole di entrambe sono descritte nella seguente tabella:

Tab.	Priorità	Match	Azione	Quando viene creata
0	200	ipv4_src = 10.0.0.1, protocollo UDP	output:4 (verso h6)	Solo in allarme, su S1, con hard_timeout di 15 s
0	100	porta di accesso, IP + TCP	passa alla Tabella 1	All'avvio, una per porta di accesso: conta il TCP
0	50	porta di accesso (tutto il resto)	passa alla Tabella 1	All'avvio, una per porta di accesso: conta il non-TCP
0	0	qualsiasi pacchetto	passa alla Tabella 1	All'avvio
1	1	porta di ingresso + MAC di destinazione	output sulla porta appresa	A richiesta, dal learning
1	0	qualsiasi pacchetto	invia al controller (table-miss)	All'avvio

In concreto, all'avvio il controller installa in Tabella 0, per ogni porta collegata a un host, due regole di sola misura (priorità 100 per il TCP, 50 per il resto) e una regola di default (priorità 0) che passa il pacchetto alla Tabella 1. In caso di allarme aggiunge su S1 una regola a priorità 200, con match sull'IP sorgente dell'attaccante e sul protocollo UDP, che redirige i pacchetti verso h6.

La Tabella di seguito riassume invece i parametri che regolano il comportamento del controller, insieme alle ragioni che hanno guidato la loro scelta. Il traffico viene campionato ogni 2 secondi: la soglia e il numero di campioni consecutivi stabiliscono quando un host è considerato un attaccante, mentre durata del blocco e riblocco rapido definiscono come si reagisce e con quale tempestività. Chiudono la tabella l'azione di mitigazione adottata e gli stati con cui il controller tiene traccia di ogni porta.

Parametro	Valore	Motivazione
THRESHOLD_NON_TCP	200 000 B/s (≈1.6 Mbit/s)	Sopra il traffico dei legittimi e sotto il flood di h1; h5 (≈2.11 Mbit/s) la supera ma è in whitelist
CONSECUTIVE_NON_TCP	3 campioni (≈6 s)	Isteresi: evita falsi positivi su micro-burst
BLOCK_HARD_TIMEOUT	15 s	Sblocco gestito dallo switch (hard_timeout), senza timer nel controller
FAST_REBLOCK	attivo	Il recidivo viene ribloccato dopo 1 solo campione sopra soglia
Azione di mitigazione	Redirect verso h6 (output:4 su S1)	Isola l'attacco e conserva le prove, invece del semplice DROP
Stati per porta	NORMALE → ALLARME → BLOCCATO	Registrati in monitor_mitigation.csv

4. Scenario senza difese: baseline e attacco

Nello scenario senza difese il controller (un semplice *simple_switch13* di Ryu) svolge solo il forwarding, senza alcun monitoraggio. L'esperimento si articola in quattro momenti, presi a riferimento dall'istante t = 0 s in cui partono i client TCP.

1. Al primo momento, a $t = 0\ s$, h2 e h4 avviano le trasmissioni TCP verso h3 (porte *5002* e *5003*), con un throughput di circa *1.05 Mbit/s* ciascuno.
2. A $t \approx 6\ s$ si aggiunge h5, che trasmette UDP a 2.11 Mbit/s verso la porta 5004. Fino a $t \approx 25.4\ s$ la rete funziona in condizioni normali e questa fase costituisce la baseline di confronto.
3. A $t \approx 25.4\ s$ h1 avvia il flood UDP verso la porta 5001. Da quel momento il link S3–S4 risulta saturo: il TCP di h2 e h4 rileva le perdite e riduce progressivamente la propria finestra di congestione fino ad andare in timeout, così il throughput scende da *1.05 Mbit/s* a valori prossimi a 0 (il cosiddetto fenomeno di TCP starvation). L'UDP, privo di controllo di congestione, non rallenta: h1 continua a inviare e h5 perde circa il 39% del proprio throughput rispetto alla baseline.
4. Quando il flood termina ($t \approx 63.4$) , il link si libera e il throughput di h2 e h4 risale superando per qualche secondo il valore nominale. Con il bitrate fissato (-b), iperf mantiene la velocità richiesta come media nel tempo: durante la starvation ha accumulato dati non trasmessi e, appena la congestione cessa, li recupera a raffica. Il picco è quindi un effetto di recupero e il throughput rientra sul valore nominale una volta smaltito l'arretrato.

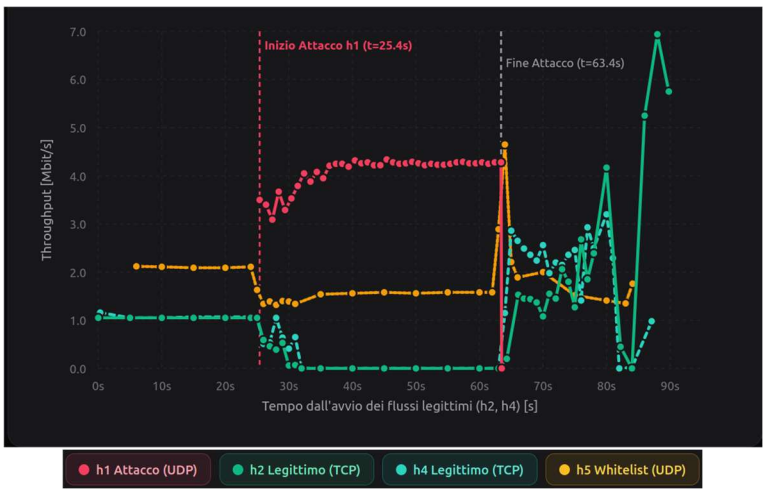


Figura 2 – Throughput di h2, h4 (TCP) e h5 (UDP) senza difese: il flood di h1 ($t \approx 25.4\ s$) azzera il TCP e riduce il throughput di h5.

6. Scenario con difese: mitigazione reattiva e quarantena forense

Nello scenario con difese il controller completo monitora ogni porta di accesso e interviene automaticamente. Il test ripete le condizioni precedenti:

1. h2 e h4 avviano le trasmissioni TCP a $t = 0\ s$ e h5 si aggiunge a $t \approx 6\ s$, così la rete lavora in condizioni normali fino all'attacco.
2. A $t \approx 26\ s$ h1 avvia il flood UDP. Il controller legge i contatori ogni 2 secondi e rileva tre campioni consecutivi ($t = 26, 28, 30\ s$) sopra la soglia sulla porta 1 di S1;
3. A $t \approx 30\ s$ lo stato passa a *ALLARME*. Subito dopo installa su S1 una regola a priorità 200, con match sull'IP sorgente di h1 e sul protocollo UDP, che ne rediretta i pacchetti verso h6, e lo stato diventa *BLOCCATO*. Il traffico malevolo non attraversa più il link S3–S4. h5, pur superando la soglia, non viene toccato perché è in whitelist.
4. Con il collo di bottiglia libero, h2 e h4 recuperano il throughput nominale e h5 torna ai valori di baseline. Dopo *15 s* la regola scade tramite *l'hard_timeout* dello switch: se h1 continua l'attacco, *FAST_REBLOCK* lo blocca di nuovo al primo campione sopra soglia, altrimenti la porta torna allo stato *NORMALE*.

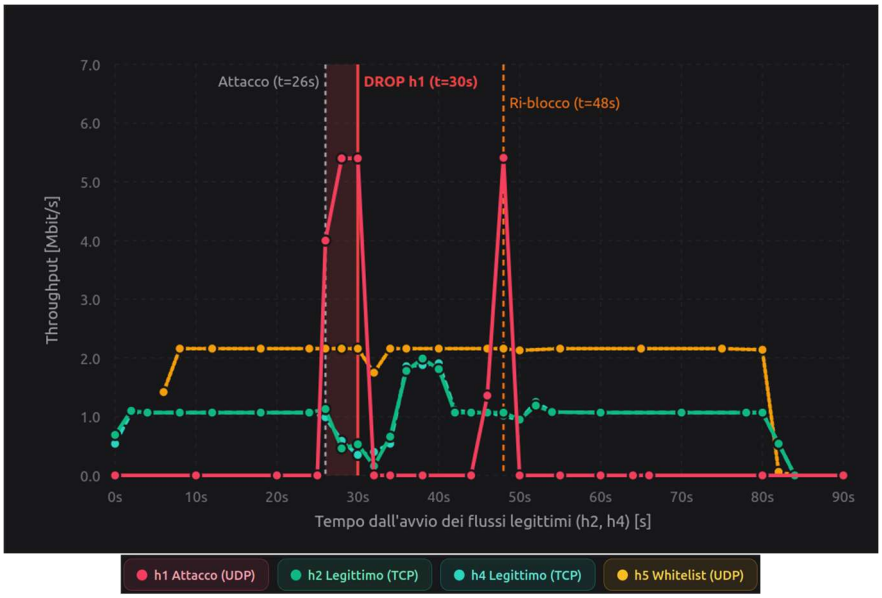


Figura 3 – Throughput di h2, h4 (TCP) e h5 (UDP) con difese: il flood di h1 ($t \approx 26\ s$) viene rilevato a $t \approx 30\ s$ e rediretto verso h6, e i tre flussi tornano ai valori di baseline.

6.1 Quarantena forense

Nella fase di mitigazione il traffico di h1 viene rediretto verso un host di quarantena, h6, invece di essere scartato. La scelta ha tre motivi:

- Il primo è conservare le prove: con un semplice DROP l'attacco sparirebbe senza lasciare tracce, mentre su h6 tcpdump registra tutto in formato .pcap, leggibile con Wireshark.
- Il secondo è poter verificare la decisione del controller: avendo sottomano il traffico bloccato, si può controllare che fosse davvero malevolo e, in caso contrario, correggere soglia e parametri.
- Il terzo è l'isolamento: i pacchetti non raggiungono più il collo di bottiglia S3–S4, ma percorrono il solo tratto h1–S1–h6; quindi, la rete di produzione non risente della cattura.

La regola modifica soltanto la porta di uscita, perciò i pacchetti arrivano a h6 con le intestazioni originali (10.0.0.1 → 10.0.0.3:5001, payload di 1470 B). Questo permette di ricostruire la firma dell'attacco: sorgente, destinazione, protocollo, dimensione e cadenza dei pacchetti.

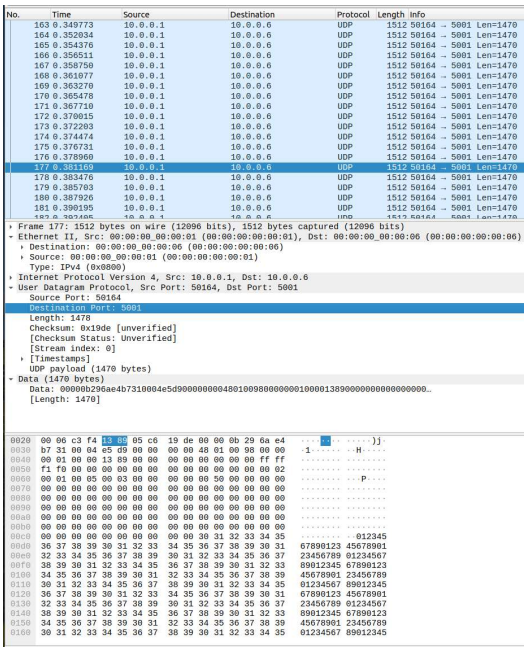


Figura 4 – Cattura su h6 in Wireshark: pacchetti UDP da 10.0.0.1 a 10.0.0.3:5001, con payload di 1470 B, rediretti dal controller sulla porta 4 di S1.

7. Conclusioni e risultati

In conclusione, i risultati confermano l'efficacia della mitigazione:

- Senza difese il TCP di h2 e h4 resta azzerato per tutta la durata dell'attacco (circa 38 s) e h5 perde fino al 39% dei datagrammi (26.8% in media).
- Con il controller il disservizio si riduce a circa 4 s, il tempo necessario a raccogliere i tre campioni consecutivi sopra soglia, la perdita di h5 resta trascurabile e il throughput di h2 e h4 torna ai valori di baseline (la media supera il valore nominale per l'effetto di recupero di iperf descritto nella sezione 5).
- Alla scadenza del blocco, se l'attacco prosegue, FAST_REBLOCK ripristina la mitigazione in circa 2 s, come si vede dalla perdita del 100% di h1 nel grafico. Un blocco mirato sull'attaccante protegge quindi il traffico legittimo, h5 compreso, senza interventi manuali.

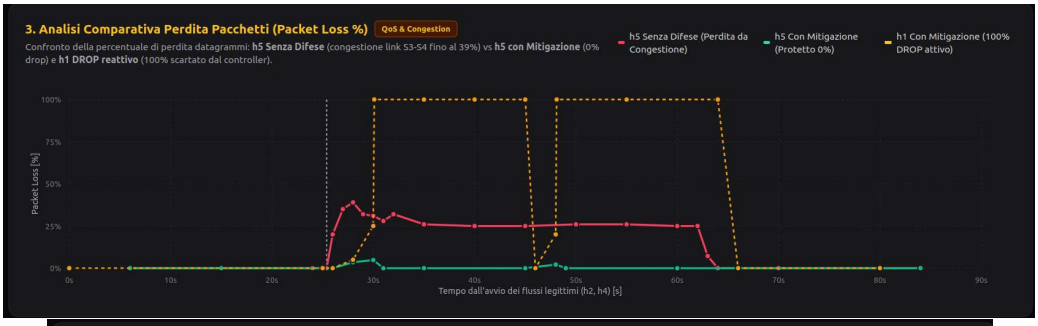


Tabella 1: Sintesi Throughput & Disservizio (Metrica Originale)

Valori di banda trasferita, picchi di attacco, collasso TCP e durata del disservizio

Throughput Analysis

SCENARIO	H1 PICCO ATTACCO	H2 TCP MINIMO	H2 TCP MEDIO	H4 TCP MINIMO	H5 UDP MEDIO	DURATA DISSERVIZIO
Senza Difese (Fase 4)	4.34 Mbit/s	0.00 Mbit/s	0.00 Mbit/s	0.00 Mbit/s	1.48 Mbit/s	38.0 s (Permanente fino a fine DoS)
Con Mitigazione (Fasi 7-9)	5.40 Mbit/s	0.16 Mbit/s	1.48 Mbit/s	0.35 Mbit/s	2.16 Mbit/s	4.0 s (3 intervalli allarme)

Tabella 2: Analisi Qualità del Servizio & Packet Loss (QoS)

Metriche di affidabilità datagrammi UDP, efficacia delle flow rules di DROP e tempi di allarme

QoS & Loss Analysis

SCENARIO	INIZIO ATTACCO H1	THROUGHPUT H2 (MIN / MEDIO)	PACKET LOSS MASSIMO H5	PACKET LOSS MEDIO H5	EFFICACIA DROP H1	TEMPO DI REAZIONE CONTROLLER
Senza Difese (Fase 4)	t = +25.4 s	0.00 / 0.00 Mbit/s	39.0 %	26.8 % (durante l'attacco)	0% (inesistente)	Nessuna reazione (Disservizio 38s)
Con Mitigazione (Fasi 7-9)	t = +26.0 s	0.16 / 1.48 Mbit/s	0.0 % (invariato)	0.0 % (invariato)	100 % (Drop totale h1)	4.0 s (Sforamento 3/3)